1940:

Second World War: Four days after the French government fled Paris, German forces occupied the French capital, a major accomplishment in the Fall Rot operation. https://en.wikipedia.org/wiki/Fall_Rot

1966:

The Vatican formally abolished its 427-year-old list of prohibited books (title page pictured). https://en.wikipedia.org/wiki/Index_Librorum_Prohibitorum

1996:

After an 81-day standoff sparked by their refusal to be evicted from their foreclosed property in Jordan, Montana, the Christian Patriot group Montana Freemen surrendered to the FBI. https://en.wikipedia.org/wiki/Montana_Freemen

---

Wiktionary's word of the day:

quotationist: 1. One who makes, or is given to making, quotations; a quoter. 2. About Word of the Day 3. Nominate a word 4. Leave feedback https://en.wiktionary.org/wiki/quotationist

---

Wikiquote quote of the day:

” --Donald Trump https://en.wikiquote.org/wiki/Donald_Trump

FRED TALKS

14th June 2026

CONTENTS

LLM Judges aren’t the shortcut you think

DOUG TURNBULL · 03 NOV 2025 · SOURCE

Is “LLM as a judge” overhyped? After years of implementing LLM judges for clients, I find more of them recognize the limitations.

LLM judges fail for the same reason any evaluation method fails: teams lack good data+evaluation to begin with. LLM judges aren’t the shortcut teams hope. Instead, they are themselves a thing that needs data and monitoring to get use out of.

Let me walk through the pitfalls of LLM as a judge (for search) that you’ll encounter.

LLM as a judge

When I talk about LLM as a judge, my focus is search. I mean creating a judgment list. For search that means labeling how relevant a document is for a query. How relevant is the product “garden trowel” for a `q=shovels`? Maybe this “garden trowel” is less relevant than “iron spade”?

In the ancient past, to improve the search, I would be limited to a tiny hand-labeled judgment list. Labeling by hand is expensive. Datasets drift out of date quickly. New use cases arise. So we constantly need humans to relabel.

These costs inspire teams to use LLMs to build judgments, They:

1. Start with a ground truth of human labels
2. Build a prompt to approximate that ground truth
3. Apply those prompts to go beyond the ground truth to new query / documents

If done well, we can sit at our laptop, tinker with relevance algorithms, and continue to tune without bother humans for non-stop relabeling.

Sounds promising. But teams often ignore many of the assumptions and nuance underlying the task of LLM labeling. Let’s go through them, and we’ll see at the end, maybe LLMs should play a different role in evaluation?

reporting back to the parent agent, then it will be missing from the context. In this way, the Anthropic models can more easily 'miss' obvious facts, even if it seemed like they at one point had done the research. This can happen with compaction too, but it's less likely, because the model is doing 'less work' to omit or preserve tokens.

In the end state, I expect we will see **both** approaches combined. Anthropic will improve their compaction (which right now is too lossy), and OpenAI will train for multi-agent setups.

[Daily article] June 14: Early life and education of Donald Trump

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 14 JUN 2026 · SOURCE

The early life of Donald Trump, the 45th and 47th president of the United States, began with his birth on June 14, 1946, in the borough of Queens in New York City. Donald Trump's father was Fred Trump, a real- estate developer; his mother Mary Anne Trump was a Scottish immigrant. Donald was enrolled at age five at the Kew-Forest School, a private school in Queens. When Donald was in seventh grade, Fred discovered that his son was secretly going into Manhattan to obtain knives. Donald was sent to the New York Military Academy (NYMA); he graduated in May 1964. Trump attended Fordham University from 1964 to 1966, studying economics. His college enrollment—and later a medical exemption—allowed him to defer the Vietnam War draft. In his sophomore year, seeking a larger business network, Trump applied to transfer to the Wharton School of the University of Pennsylvania, an Ivy League school favored by his father. Trump graduated from Penn in May 1968 with a Bachelor of Science in economics. (Full article...).

Read more: https://en.wikipedia.org/wiki/Early_life_and_education_of_Donald_Trump

---

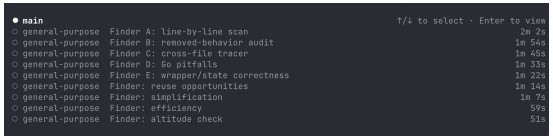
Today's selected anniversaries:

1846:

Settlers in Sonoma began rebelling against Mexico, later proclaiming the California Republic and raising a homemade flag with a bear and a star. https://en.wikipedia.org/wiki/California_Republic

Since at least Opus 4.1, I've noticed Claude Code will eagerly take this approach. For any research of the codebase, the model will invoke `Explore` sub-agents which leverage Haiku to create a quick summary.

And when running Fable 5, it goes *nuts*.



This is very different approach: sub-agents are now able to do large amounts of work within a context window, and then pass back only the relevant information to the parent agent.

In effect, this looks *more* like the way human organizations run. Everyone has their set of goals and inputs and outputs. They see some subset of the total information available, and make decisions based upon that. And we all communicate via language. But we don't get to see the hidden state in one another's heads.

Claude models *also* compact, and it takes advantage of a few of these different approaches. But the compaction is slow and requires users to upgrade the client consistently, so I assume the training regime has leaned more on delegating to sub-agents.

Takeaways

Cost and token efficiency: I suspect that Anthropic models tend to cost more because the sub-agents often end up doing duplicate work. They may be searching similar files because they aren't actively communicating.

	MODEL	SCORE	COST / TASK	TOKENS / TASK	STEPS / TASK
1	Fable 5 Max ●	72.9%	$18.02	63,842	76
2	Fable 5 Extra High ●	72.0%	$13.74	48,754	63
3	Fable 5 High ●	70.6%	$10.81	37,173	54
4	Fable 5 Medium ●	69.8%	$8.27	28,507	47
5	Opus 4.7 Max ○	64.8%	$11.02	62,989	96
6	GPT-5.5 Extra High ●	64.3%	$4.37	17,905	46

Perceived speed: Anthropic models will often seem to be "doing more", because the tokens are being produced in parallel vs serial. Many more tokens are produced during that time.

'**Forgetting':** a friend pointed me to some cases where Anthropic models seem less coherent, or more often tend to misreport facts to the user. I think this is true, and it's easily explained with the message-passing approach taken by sub-agents. If a sub-agent deemed a fact was not worth

Problem one - LLM evals don’t know what’s engaging to users

Users’ ever-evolving lizard brains drive conversions. And while optimizing for just lizard brains would itself be a nightmare, so would only focusing on an LLM’s opinion.

LLMs do a better job approximating *human evaluations*. In search, we call these evaluations *explicit* judgments. They use human labelers’ basic knowledge of the world to label a result as *topically relevant* to the query or not.

Topical relevance is only one kind of relevance. As an example, for a search of “articles about harry potter”. The article is either factually about Harry Potter or its not. We might also consider other gradations:

- Is the article well written?
- Is it article from an authoritative source?
- Is the article about a tangential topic? Dumbledore vs Gandalf etc?

All of which relate to topicality, authority, and knowledge. An LLM excels at this — LLMs live in a world of facts and knowledge.

But LLMs don’t have limbic systems. So their evaluations come with limitations.

In real search applications, “relevance” goes beyond topicality. I worked at Reddit on search. A search for “harry potter” might mean anything from a memes, to drama about JK Rowling, to every kind of juicy controversy, spicy post, or oddity. People search for “cybertruck” not for reviews but to see silly videos.

Then there’s the more mundane. I found at Shopify, users preferred plain or black clothing over flashy items based on their clicks. Unless you tell an LLM this (because you’ve done the analysis) an LLM won’t intrinsically catch this.

Often the “topical” part of search is relatively easy. But engaging search, that captures these human oddities, must go beyond world-knowledge to at least get *some* of a user’s lizard brain.

Problem two - the last 10% of disagreement is the important stuff

Many teams get excited when they immediately see 70-80% agreement with human labelers. With a bit of tuning, they can inch closer to 90%.

In my experience, that last bit of human-LLM disagreement matters. Those are the non-obvious cases. Search deals with a big hard negatives challenge. Those examples that deceptively seem relevant naive search algorithms (or labelers), but are, in fact irrelevant. That last 10-30% might be what more careful labelers would catch. Or represented in clickstream based labels.

For example on this furniture dataset, a LLM doesn't understand that a "bistro table" is meant for ***outdoor*** use in US furniture lingo. That's not obvious to an LLM: The LLM imagines I'm opening a restaurant.

Simple algorithms often capture that first 80-90%. It's the 10-20% you need great labels for. So pay attention not just to coverage. But for which use cases.

A 90% coverage might be good regression tests, it might not help you go beyond easier wins.

Problem three - sneaky overfitting

So far, I've mentioned a few strange cases the LLM judge might miss: a bistro table is meant for outdoor usage. Users preferring darker clothing items.

The savvy prompt engineer will want to fill the prompt with examples to help the LLM perform better on these cases.

But beware overfitting to these particular failure cases.

With every new rule, you push the model's meager attention away from the general problem, and towards your exceptions. Layer example after example, and you'll find yourself with a brittle system focused excessively on specific use cases.

Overfitting with prompts can be sneaky. Playing whack-a-mole with your eval data can degrade generality. You can get perfect results on the eval set, only to expand to unseen queries, and find the gains no longer hold.

OpenAI: the oracle

Since roughly ChatGPT 5.3-Codex, I've noticed that the model has improved a *lot* at dealing with long context windows. It stays coherent even across long-running tasks or `/goal` implementations, despite having a smaller context window than the corresponding Opus models (~200k vs 1m).

The approach Codex takes is **compaction**, and there are two naive approaches you can use:

1. you have a separate (sometimes smaller) model output a new message based upon the trajectory.
    - *e.g. ask 5.5 to summarize everything in the thread up to 1k tokens*
2. you remove certain categories of calls from the conversation.
    - *e.g. remove all tool calls, then begin inference*

You might even imagine doing this in parallel: compacting up to a certain number of messages in a thread, and then leaving more recent ones in full fidelity.

Since earlier this year, Codex has shipped native server-side compaction built into the responses API. When the produced tokens start to overflow the context window, Codex can run a process to compact everything and retain only the relevant information.

Given that the compaction happens server side, it has two nice properties: a) OpenAI can change the implementation easily and at will without affecting clients and b) the API can take advantage of much better K/V caching for long-running threads by routing to the right GPU.

The net result, I think about like an 'Oracle'. Codex typically keeps one long thread going with frequent compactions. You will see sub-agents which execute on clean paths, but it tends to happen less often, unless nudged by the user.

Because the single thread is managing everything related to user responses, it maintains a lot of coherence. Small details which are relevant to the overall trajectory are *remembered*.

Anthropic: the firm

Compaction is just one way of dealing with context windows. The other approach you could take is *splitting* context windows across various agents. In this technique, you split the problem into various sub-problems, then have each agent execute on the sub-problem within it's own context window.

Now go save Princess Peach.

References

```
@article{
    leehanchung,
    author = {Lee, Hanchung},
    title = {It's-a Me, Agentic AI},
    year = {2026},
    month = {02},
    day = {18},
    howpublished = {\url{https://leehanchung.github.io}},
    url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent
}
```

The Oracle and the Firm

CALVIN FRENCH-OWEN · 13 JUN 2026 · SOURCE

Like most of the internet, I've been diving into Fable 5 over the last 24h. And like most of the internet, I've been pretty blown away with the quality.

But as I've been using both Fable and GPT-5.5, I couldn't help but notice there are clear differences in approach which make the two models behave quite differently. And we're seeing two very different training regimes play out.

For any frontier model, accomplishing real work is an exercise in **context management**. The model needs to solve a problem across a very large number of tokens; some are explored via tool calls, others are the model thinking. Then it needs to produce a result.

To get models to solve harder and harder tasks that run for increasing amounts of time, you need to figure out how to scale that context management.

To do a good job, you need holdout data you're not tuning against. A set of judgments you don't peak at. Because peaking might cause you to sneak a change into the prompt, making the holdout less useful as an independent validation step.

Add rules and exceptions carefully, you'll be surprised how easy it is to pull an LLM away from its generality.

Problem four - LLMs only evaluate the document it sees

LLMs only evaluate the document it sees. IE:

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
```

Is this enough information for evaluating the product? Is it what a user sees?

Suddenly, you decide to add new information about the product

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
Category: Outdoors
```

What are we evaluating truly?

- The products relevance?
- The usefulness of a set of product properties for determining relevance?
- How comprehensively you're describing products to an LLM?

Really it gets down to a classic problem in search. There's perceived relevance - the relevance of the product, as it appears in your product page. Then there's actual relevance - the detailed, thoughtful understanding of your product by a user. A lot depends not just on true relevance, but how it's presented to users.

Problem five - Your LLM judge can't be trusted on novel use cases

I assume the goal of LLM as a judge in to align to human labelers. But rarely do those judgments comprehensively represent every use case.

Search metrics with human evals don't go up-and-to-the-right. They seesaw as teams reactively label, solve, label, solve ad infinitum. In my experience it's rare to find a team build a true, comprehensive human labeled evals covering every use case. Even if we did, we'd need to account for evolving search use cases.

The flow usually goes like this: we want to solve a problem (search by product name). So we capture some judgments for that use case. Relevance starts low. We work a bit to improve this use case and it goes up. Yay!

But the boss comes to our desk and suggests we have a new problem to solve: search of furniture by the room (living room, etc). We gather more labels. We know we're bad at this use case. So search metrics go back down. Sad 😟

We solve that use case, it goes back up to where it was before. Repeat. Up and down. See-saw.

This presents a problem for an LLM judge. If our human labels only focus on a subset of use cases, **we're also aligning our LLM judges on a subset of use cases.**

Our LLM judge generalizes to just a piece of the problem. The LLM judge can't be trusted on a new use case until we measure it's abilities in that use case.

So if you seesaw its human labeling, you must seesaw the alignment of the LLM judge to the use cases we capture in human eval. As we adopt a new use case, we need more human labels. We need to retest the LLM judge. And so on.

The task of human labeling does not end with LLM Judges. In fact its even more crucial.

How to LLM as a judge

Here's how I use LLM judges:

- To extend labels on use cases I already measure decently to new queries (because I can trust alignment of the LLM on this use case)
- As a safeguard against regressions

World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser's Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today's World 8 is tomorrow's World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	**Agentic AI**
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers
Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems

The next time someone asks you to explain agentic AI, ask them: have you played Mario?

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who's actually making all of this work? Mario doesn't train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what "good" looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn't one level. It's organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design

I trust the LLM judge to flag a regression on the use cases my human labels cover. The LLM is aligned and measured against these. I don't trust the LLM to discover new, unexpected ways users search. I look to the users for that, not LLMs.

Still many teams don't even do decent monitoring of topical relevance. There are plenty of obviously broken search engines. The ones that would benefit from an LLM judge, likely would benefit from rounds of human labeling first needed before blindly trusting an LLM judge.

We already have automated data labeling at home kids

If we have enough labels to evaluate LLM as a judge on a use case, we have enough to train an ML model to recover those labels. We can then see a better place LLM's fit into evaluation:

- LLMs focus on factual, topical parts of the problem. But miss other parts of the problem
- LLMs might see a subset of the content features
- Other non-LLM features might help us better evaluate relevance (embedding similarity, text matches, etc)
- Other ground-truths (besides human) might be a better definition of relevance (ie engagement based labels)

Instead of LLM as a Judge, **why not Learning to Rank as a judge?** We seem to have forgotten that ranking models provide value beyond directly deploying them to production. They can help guide more manual search tuning as well.

I'm becoming more of a fan of LLM-for-feature-generation over LLM as a judge. LLMs can cover the topical part of relevance. You can ask them "Which of these product names is more relevant to the query?". The downstream, traditional ML model is agnostic to where the features come from. They could be an embedding similarity, LLM decision, lexical similarity, or more.

That's much more my interest these days. When someone says "LLM as a judge" I nod and say yes let's build that, but then go on and probably look into using LLM features heavily in a larger model. LLMs can be very useful for rapid feature development: they're promptable and easy to play with.

If you're interested in this approach, check out this blog post and the corresponding Haystack talk. Also a talk from Shopify choosing cross encoders for evaluation over LLMs

Enjoy!

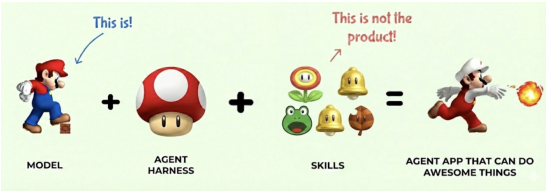
Enjoy softwaredoug in training course form!

I hope you join me at Cheat at Search with LLMs to learn how to apply LLMs to search applications. Check out this post for a sneak preview.

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · SOURCE

Agentic AI is a fairly recent development that combines reasoning (OpenAI, 2024) tool use (Schick et al, 2023) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart's Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here's where the full picture comes together. How does Mario learn to complete levels? **Reinforcement learning.**

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what's on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment
4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in Agents Are Workflows. The same loop that every agentic AI system runs:

The value of Mario's current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. “Summarize this codebase” and “refactor this codebase” use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there’s a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here’s the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Small Mario is the **base pretrained model**. He’s just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he’s dead. He can’t break bricks. He can’t take damage. He has potential, but he’s not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it’s game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn’t run off cliffs
- **Memory and context management** so it remembers where it’s been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he’s a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it’s a product.

> The Super Mushroom doesn’t change WHO Mario is. It changes what he can SURVIVE. The model harness doesn’t change the model’s core knowledge. It changes what the model can handle in production.

Now here’s where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	**Code execution** - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	**Web search** - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	**File system access** - explore and modify the codebase, reach files the model couldn't otherwise touch
Hammer Suit	Throw hammers, defeat almost anything	**Shell/terminal access** - the most powerful tool, can interact with the full system
Star	Temporary invincibility, blow through everything	**Extended thinking/reasoning mode** - brute force through complex problems at higher compute cost
Cape Feather	Sustained flight and spin attack	**MCP servers** - sustained, extensible access to external services and APIs

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools
Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts
Clouds / Platforms	Abstractions and scaffolding that support traversal
Pits	Catastrophic failures, unrecoverable errors

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.